Teste de Software

Detecção de Bad Smells e Refatoração Segura

Relatório do trabalho prático

Gustavo Azi Prehl Gama

Matrícula: 754737

Repositório: https://github.com/gustavoprehl/bad-smells-js-refactoring

Novembro de 2026

1. Análise de Smells (análise manual)

A leitura de src/ReportGenerator.js revelou um único método público, generateReport, que concentra toda a lógica do serviço. Os três smells mais relevantes encontrados foram:

- **Método Longo / condicionais aninhadas.** generateReport monta o cabeçalho, filtra os itens por perfil, formata cada linha, acumula o total e monta o rodapé, com até quatro níveis de if aninhados (for → perfil → valor → formato). Para entendê-lo é preciso simular mentalmente todas as combinações. Para os testes, o único ponto de entrada obriga a cobrir cada caminho (2 formatos × 2 perfis × faixas de valor) pelo resultado final: não é possível testar a formatação de uma linha ou a regra de visibilidade isoladamente.
- **Código Duplicado e Switch Statements (condicionais por tipo).** a verificação reportType === 'CSV' / 'HTML' aparece em cinco pontos; o template da linha CSV está copiado nos ramos ADMIN e USER e total += item.value se repete quatro vezes. Adicionar um formato novo (ex.: PDF) exige alterar três blocos diferentes (cirurgia com espingarda). A duplicação já gerou um defeito: as duas cópias da linha HTML divergiram e a versão do admin produz "<tr >" (com espaço) para itens não prioritários — exatamente o caso que a suíte de testes acusa.
- **Números e Strings Mágicos.** os limites 1000 (prioridade) e 500 (visibilidade do usuário comum) e as strings 'ADMIN', 'USER', 'CSV' e 'HTML' estão soltos no código, sem nome que explique a regra de negócio. Uma mudança de regra exige caçar literais, e os testes de borda (500/501, 1000/1001) dependem de conhecer valores que não estão documentados em lugar nenhum além de um comentário.

Também foram observados: o método altera os itens recebidos (item.priority = true), um efeito colateral invisível para quem chama, e o atributo db, que é recebido mas nunca usado (generalidade especulativa).

2. Relatório da Ferramenta (ESLint + eslint-plugin-sonarjs)

O projeto foi analisado com ESLint 8 e eslint-plugin-sonarjs 0.21, usando o .eslintrc.json com "plugin:sonarjs/recommended" e o limite de complexidade cognitiva do repositório (5, mais rigoroso que o 15 sugerido no enunciado). Resultado de npx eslint src/ antes da refatoração:

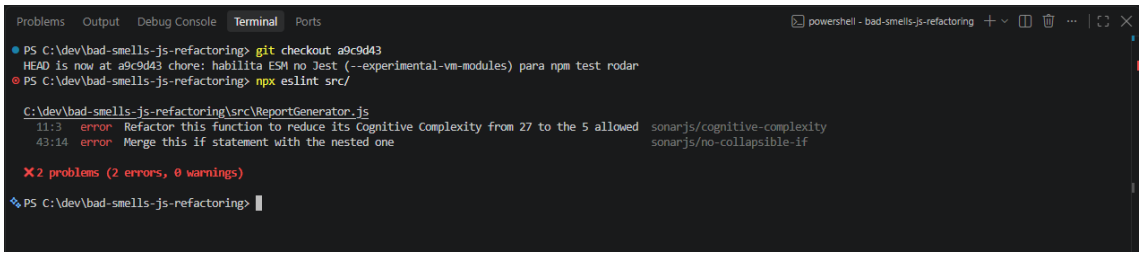


Figura 1 — npx eslint src/ no código original: 2 erros.

O plugin agregou o que a leitura manual não consegue medir:

- **Complexidade cognitiva 27:** a análise manual dizia apenas que o método era "complicado"; a ferramenta quantificou o problema (27, contra o limite de 5) e deu um critério objetivo de término para a refatoração.
- **no-collapsible-if (linha 43):** o else if (USER) seguido de if (valor <= 500) podia ser fundido em uma única condição. É um detalhe estrutural fácil de passar despercebido na leitura.

Por outro lado, a ferramenta não apontou os números mágicos nem os templates duplicados (as strings aparecem só duas vezes, abaixo do limiar da regra no-duplicate-string). Por isso, a análise manual e a automática se complementam.

3. Processo de Refatoração

O smell mais crítico corrigido foi o Método Longo com condicionais aninhadas, já que ele concentra a complexidade e é a origem da duplicação. A refatoração foi feita em src/ReportGenerator.refactored.js, em passos pequenos, com npm test e ESLint executados após cada um.

Antes — corpo de generateReport (trecho, original)

```
for (const item of items) {
  if (user.role === 'ADMIN') {
    if (item.value > 1000) {
      item.priority = true;
    }
    if (reportType === 'CSV') {
      report += `${item.id},${item.name},${item.value},${user.name}\n`;
      total += item.value;
    } else if (reportType === 'HTML') {
      const style = item.priority ? 'style="font-weight:bold;"' : '';
      report += `<tr ${style}><td>${item.id}</td><td>${item.name}</td>...</tr>\n`;
      total += item.value;
    }
  } else if (user.role === 'USER') {
    if (item.value <= 500) {
      if (reportType === 'CSV') {
        report += `${item.id},${item.name},${item.value},${user.name}\n`;
        total += item.value;
      } else if (reportType === 'HTML') {
        report += `<tr><td>${item.id}</td><td>${item.name}</td>...</tr>\n`;
        total += item.value;
      }
    }
  }
}
```

Depois — generateReport e as políticas por perfil (refatorado)

```
const ROLE_POLICIES = new Map([
  [ROLES.ADMIN, { canView: () => true,
                  isHighlighted: (item) => item.value > PRIORITY_THRESHOLD }],
  [ROLES.USER,  { canView: (item) => item.value <= USER_VALUE_LIMIT,
                  isHighlighted: () => false }],
]);

generateReport(reportType, user, items) {
  const formatter = FORMATTERS.get(reportType);       // CsvFormatter | HtmlFormatter
  if (!formatter) {
    return '';
  }
  const policy = ROLE_POLICIES.get(user.role) ?? NO_ACCESS_POLICY;
  const visibleItems = items.filter((item) => policy.canView(item));

  const report =
    formatter.header(user) +
    this.buildRows(formatter, policy, user, visibleItems) +
    formatter.footer(sumValues(visibleItems));
  return report.trim();
}
```

Técnicas aplicadas:

- **Replace Conditional with Polymorphism:** os três blocos if CSV / else if HTML (cabeçalho, linha, rodapé) viraram as classes CsvFormatter e HtmlFormatter, com a mesma interface header/row/footer. Um formato novo passa a ser apenas uma classe nova.
- **Decompose Conditional:** as regras de cada perfil foram extraídas para ROLE_POLICIES (canView / isHighlighted), eliminando os ifs aninhados e o no-collapsible-if.
- **Extract Method:** buildRows e sumValues isolam a montagem das linhas e o cálculo do total. O total += item.value repetido virou um único reduce sobre os itens visíveis.
- **Replace Magic Number with Symbolic Constant:** PRIORITY_THRESHOLD, USER_VALUE_LIMIT, ROLES e REPORT_FORMATS.

Resultado: a complexidade cognitiva caiu de 27 para no máximo 5 em todas as funções, e o ESLint não reporta nenhum problema no arquivo refatorado (Figura 2). Como a linha HTML passou a ter uma única implementação, o defeito do "<tr >" desapareceu, e o destaque deixou de alterar os itens recebidos.

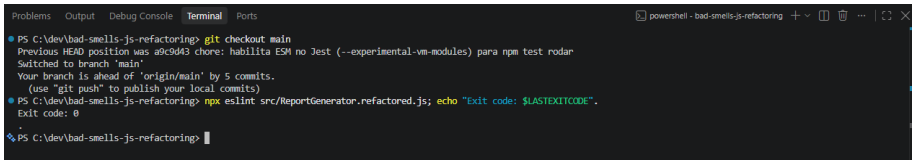


Figura 2 — ESLint no arquivo refatorado: nenhum problema (exit code 0).

4. Uso da Rede de Segurança

Antes de refatorar, foi necessário habilitar ESM no Jest (script test com --experimental-vm-modules), pois o npm test original não executava. Os testes não foram alterados. A primeira execução já revelou um defeito pré-existente: 4 de 5 testes passam no código original, e falha o HTML do admin por causa do "<tr >" (Figura 3). Seguindo o enunciado, o arquivo original foi mantido intacto e o defeito ficou documentado.

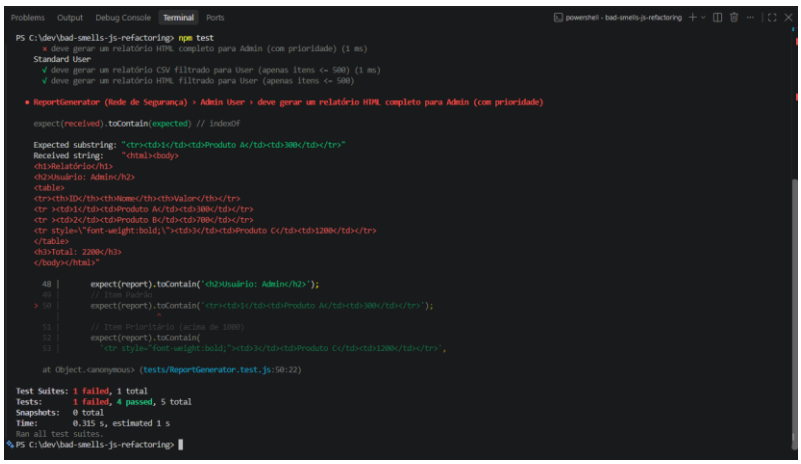


Figura 3 — npm test antes da refatoração: falha pré-existente no código original.

A cópia do teste (tests/ReportGenerator.refactored.test.js) difere do original apenas no import. Cada passo da refatoração foi um commit separado (Figura 5), sempre com os testes executados. Ao final, a suíte refatorada passa 5/5, e a do original continua acusando apenas o defeito conhecido (Figura 4). Como verificação adicional, as saídas do original e do refatorado foram comparadas em 24 cenários fora da suíte (limites 500/501 e 1000/1001, formato e perfil desconhecidos, lista vazia): todas idênticas, exceto pelo defeito corrigido.

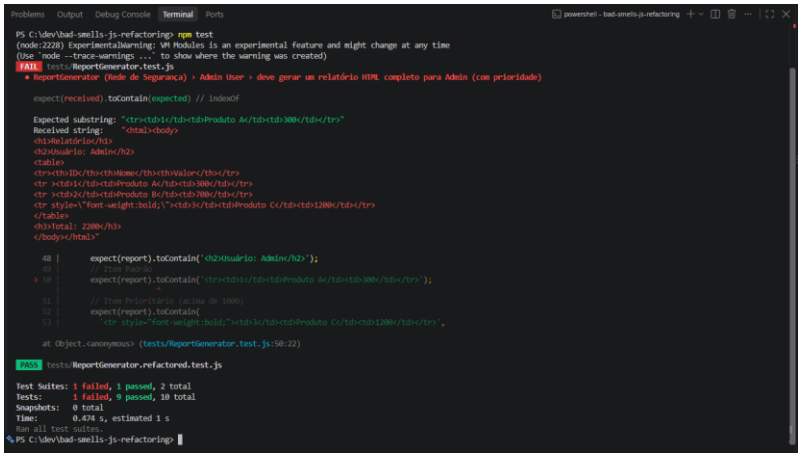


Figura 4 — npm test final: refatorado 5/5 (PASS); original falha só no defeito conhecido.

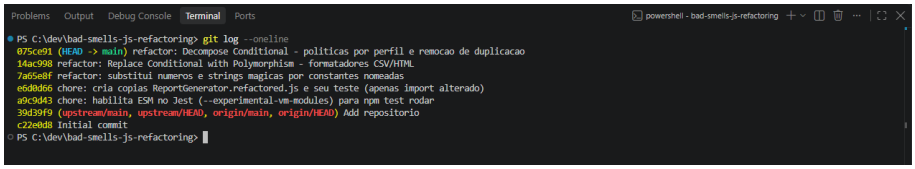


Figura 5 — histórico de commits: refatoração incremental.

5. Conclusão

A suíte de testes foi o que tornou a refatoração segura: como ela verifica o comportamento e não a implementação, o código pôde ser reestruturado por completo, com uma resposta imediata a cada passo. Ela também mostrou seu valor antes da primeira linha alterada, ao acusar um defeito que a leitura do código não havia evidenciado. A redução dos smells trouxe ganhos concretos: o método principal agora se lê como a descrição da regra de negócio, cada responsabilidade (formatação, visibilidade, total) pode ser testada e alterada isoladamente, e a inclusão de um novo formato ou perfil não exige tocar no código existente. O resultado é um software mais fácil de entender, manter e testar, que é o objetivo final de eliminar bad smells.